Computer Programming Lab:9

Introduction to functions in C++

Prepared by:

Name: Hikmat Ullah Jan

Department: Electrical Comm

Semester: 2nd

Section: B

Reg No: BF25WNELE0662

University of Engineering& Technology Peshawar

Objectives

The objective of this lab is to:

- Introduction to Function
- Prototype of Function
- Examples

Introduction

Functions let you chop up a long program into named sections so that the sections can be reused throughout the program. Function has a name identifier, accepts parameters and returns a result. C++ functions can accept an unlimited number of parameters. In general, C++ does not care in what order you put your functions in the program, so long as the function name is known to the compiler before it is called. We have been consistently using library functions in the programs such as getch() and clrscr(). We just simply use these functions without knowing what's inside these functions. We just call them from our program with some parameters inside the brackets and it does the work for us. We don't know its implementation. When we actually call these functions, the code inside these functions executes and at the end it transfers the control back to the program from where the function was called. In this lab we would learn how to make our own functions which can perform certain tasks. Function is differentiated from a simple variable, by the parenthesis just after the name as main(). It just indicates to the compiler that it's a function. The most common function that you have encountered is main() which you have been implementing throughout the lab. main() is the function whose name identifier is main with no parameters passed to it. Anything inside brackets are the parameters passed to the function. In case of multiple parameters, each of them are separated with a comma(,) in between. Parameters are written as the comma separated list within brackets just after the name of function. In main() function there are no parameters passed to it as the brackets after the name is empty. As you know when we use int main() in our program then we return an integer from the main function. Similarly when we define our own functions in the program, that function should return appropriate value according to the data type which is given before the name of function when we give its definition or implementation, just as we return an integer when we use int main() and nothing in case of void main(). C++ assumes that every function will return a value. If the programmer wants a return value, this is achieved using the return statement. If no return value is required, none should be used when calling the function.

Protype of Function

The prototype of a function is a way of describing the parameters (also called as arguments of function) and parameter types with which a legal call to the function can be made. It contains the name of the function, its parameters and their type, and the return value.

Syntax:

```
returnType functionName(List of parameters separated by commas);
```

Example:

```
int main();

float divide(int a, int b);

void square(float x);
```

Here names of the functions are main, divide and square which take 0, 2 and 1 param- eters respectively.

Example:

In this example main function just calls the function named asterisks which just prints the line of ten asterisks.

```
#include using namespace std;
void asterisk();   // function prototype
int main() {
 asterisk(); // function call
 return 0}
// function definition
void asterisk()
{ for (int i = 0; i < 10; i++) { cout << "*"; } }
```

The asteriks() function in the previous example does not take any argument and does not return anything represented by empty braces () after function name and void before function name respectively.

Example:

Lets go one step ahead, function asterisks (int a) with a single argument.

```
#include using namespace std;
void asterisk(int n);
// function prototype
int main() {
 asterisk(12); // function call
return 0; }

// function definition
void asterisk(int num)
{
  for (int i = 0; i < num; i++)
   { 15 cout << "*"; 16 } 17
}
```

This function would take an integer number as an argument and prints the number of asterisks equal to the number given as argument to the function in straight line. As it is given in function prototype, compiler expects function definition should take single argument which should be of integer data type. Due to this in function calling you need to give an integer as argument (parameter) of a function.

Example:

Now an example of a function with two parameters.

```
#include
using namespace std;
int add(int, int);
// function prototype
int main() {
int result;
// function call
result = add(3, 5);
cout << result;
return 0; 14 }
// function definition
int add(int num1, int num2) {
 return num1 + num2;}
```

Lab Tasks:

Task 1: Function Introduction and Declaration (greet)

```
#include <iostream>                    // Include input-output library
using namespace std;                   // Use standard namespace

void greet() {                         // Declare function with no parameters
                                          and no return
cout << "Welcome to C++ Programming Lab!" << endl; // Display greeting message
}
int main() {                           // Main function starts
greet();                               // Call the greet function
return 0;
}       cout << endl;
   }
   return 0;                           // End of program
}
```

Task 2: Basic Function Declaration and Call (sayHello)

```
#include <iostream>                    // Include input-output library
using namespace std;                   // Use standard namespace

void sayHello() {                      // Declare function
   cout << "Hello, World!" << endl;  // Print message
}
int main() {                           // Main function
   sayHello();                         // Call function
   return 0;                           // End program
}
```

Task 3: Array Input, Function for Sum & Average

```
#include <iostream>                    // Include input-output library

using namespace std;                   // Use standard namespace

int calculateSum(int arr[], int size) {  // Function to calculate sum

   int sum = 0;                        // Initialize sum variable

   for(int i = 0; i < size; i++) {     // Loop through array

      sum += arr[i];                   // Add each element to sum

   }

   return sum;                         // Return total sum

}

int main() {                           // Main function

   int arr[10];                        // Declare array of size 10

   int sum;                            // Variable to store sum

   float average;                      // Variable to store average
```

```
    cout << "Enter 10 numbers:\n";          // Prompt user
    for(int i = 0; i < 10; i++) {           // Loop for input
        cin >> arr[i];                      // Store input in array
    }
    sum = calculateSum(arr, 10);            // Call function and store resul
    average = (float)sum / 10;              // Calculate average
    cout << "Sum = " << sum << endl;        // Display sum
    cout << "Average = " << average << endl; // Display average
    return 0;                               // End program
}
```